

CLAUDE CODE DEEP DIVE WORKSHOP

Chapter 6 - Agent SDK

TypeScript, Python으로 만드는 프로덕션 에이전트

Update : 2026.09 / Claude Agent SDK 최신 기준,
공식 문서 code.claude.com/docs/en/agent-sdk 반영

Choi, WooHyung

Prin. Solutions Architect

AWS Korea

whchoi@amazon.com

Chapter 6 Agenda

9 Parts / 96 Slides

PART 01	SDK 기본	설치, query 루프, options, 권한 모드 / 11p
PART 02	쿼리와 멀티턴	스트리밍, 단일 수집, 세션 이어가기 / 10p
PART 03	커스텀 도구	인프로세스 MCP, 스키마, 에러 규약 / 13p
PART 04	구조화 출력	output_format, Zod, Pydantic / 8p
PART 05	권한과 훅	4수단 지형, canUseTool, SDK 훅 / 9p
PART 06	세션, 상태, CC 기능	영속, 비용, 스킬과 서브 에이전트 로드 / 10p
PART 07	호스팅과 프로덕션	컨테이너, 스케일, 관측, 격리 / 10p
PART 08	실전 프로젝트	사내 위키 Q&A 에이전트 서비스 / 8p
PART 09	Recap & Labs	요약, 실습 4종, 워크샵 총정리 / 13p

Chapter 6 Learning Objectives

본 챕터에서 달성할 학습 목표

AT THE END OF THIS CHAPTER

query 루프 위에 내 함수를 도구로 꽂고, 스키마 계약으로 결과를 받으며, 권한과 흑으로 통제하고, 컨테이너에 실어 운영까지 할 수 있습니다.

OBJ 1

루프 장악

query와 메시지 스트림,
options 체계 이해.

OBJ 2

도구 제작

인프로세스 MCP로 사내 시
스템을 도구화.

OBJ 3

계약 출력

Zod, Pydantic 스키마로
타입 세이프 결과.

OBJ 4

프로덕션

권한, 세션, 호스팅으로 서비
스 완성.

9 Parts

구조화된 학습 경로

4 Labs

Chapter 6 실습 랩

TS + Python

양 언어 병행

~3h + 1.5h

강의와 실습 시간

왜 SDK인가

직접 조립의 시대는 끝났다

저수준 API 직접 조립 (과거)

1. 메시지 배열, tool_use 루프 수작업
2. 재시도, 컨텍스트, 캐싱 직접 구현
3. 권한, 파일 도구도 전부 자작
4. 코드의 8할이 파이프라인 작업

Agent SDK (2026)

1. query 한 함수가 에이전틱 루프 전체
2. 도구 실행, 재시도, 컨텍스트를 SDK가 관리
3. Claude Code의 도구, 권한, 혹은 그대로
4. 코드의 8할이 내 도메인 로직

-p의 엔진

Ch.5와 동일 경로

배관 제거

SDK의 가치

자산 재사용

Ch.1-4 그대로

TS, Python

공식 양 언어

06

Agent SDK

Claude Code를 라이브러리로

CHAPTER 06 / PART 01

SDK 기본

query 한 함수에 담긴 에이전트

1. 제품 지형과 설치, 인증
2. 첫 에이전트: TS와 Python
3. 메시지 스트림과 options 지도
4. SDK 권한 모드 5종

Agent SDK란

제품 지형 속의 자리

CLAUDE CODE AS A LIBRARY

Agent SDK는 Claude Code의 에이전트 하네스를 라이브러리로 노출한 것입니다.

대화형 CLI, 헤드리스 -p, SDK 셋이 같은 엔진의 세 창구입니다.

SAME ENGINE

동일 엔진

도구, 권한, 혹은 세션이 CLI와 동일 체계.

SURFACE

세 창구

대화형(Ch.1-4), -p(Ch.5), SDK(오늘).

PACKAGES

패키지

TS @anthropic-ai/claude-agent-sdk, Python claude-agent-sdk.

RENAME

이름 변천

구 Claude Code SDK가 Agent SDK로 개명, 마이그레이션 가이드 제공.

1 엔진

3 창구

Node 18+

TS 요구

Python 3.10+

Python 요구

네이티브 동봉

TS는 CC 바이너리 포함

설치와 인증

5분 셋업

● ● ● setup

```
# TypeScript
$ npm install @anthropic-ai/claude-agent-sdk

# Python (uv 권장 / pip)
$ uv init && uv add claude-agent-sdk
$ pip3 install claude-agent-sdk

# 인증: .env
ANTHROPIC_API_KEY=...      # Console 발급 키
# 또는 CLAUDE_CODE_USE_BEDROCK=1 + AWS 자격 (Ch.3)
# 주의: claude.ai 구독 로그인은 서드파티 제품 제공 불가
```

npm / uv
설치 2줄

API 키
기본 인증

Bedrock env
USE_BEDROCK=1

정책 주의
구독 로그인 제공 금지

첫 에이전트 / TypeScript

버그를 스스로 고치는 15줄

agent.ts

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "utils.py의 크래시 버그를 찾아 수정해",
  options: {
    allowedTools: ["Read", "Edit", "Glob"],
    permissionMode: "acceptEdits"
  }
})) {
  if (message.type === "result")
    console.log("Done:", message.subtype);
} // 실행: npx tsx agent.ts
```

query

단 하나의 진입점

async iterator

for await 소비

allowedTools

능력의 선언

acceptEdits

편집 자동 승인

첫 에이전트 / Python

같은 구조, 파이썬 문법

agent.py

```
import asyncio
from claude_agent_sdk import (
    query, ClaudeAgentOptions, ResultMessage)
async def main():
    async for message in query(
        prompt="utils.py의 크래시 버그를 찾아 수정해",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Edit", "Glob"],
            permission_mode="acceptEdits")):
        if isinstance(message, ResultMessage):
            print("Done:", message.subtype)
    asyncio.run(main())
```

async for

동일 소비 패턴

ClaudeAgentOptions

옵션 데이터클래스

snake_case

명명 관례 차이

isinstance

메시지 타입 분기

메시지 스트림 해부

루프가 흘려보내는 것들

system (init)	세션 시작, 모델과 도구 구성	첫 메시지
assistant	text 블록(사고)과 tool_use 블록(호출)	content 순회
user (tool_result)	도구 실행 결과 회신	진행 관찰
result	subtype, result, usage, session_id	최종 봉투
필터링 관례	assistant의 text와 result만 표시	잡음 제거
Ch.5 대응	stream-json 이벤트와 동형	-p와 같은 강

options 지도

설정 하나로 에이전트가 정의된다

능력	allowedTools, disallowedTools, tools, mcpServers	P3, P5
감독	permissionMode, canUseTool, hooks	P5
정체성	systemPrompt (preset, append), agents	P2, P6
출력	outputFormat (json_schema)	P4
세션	continue, resume, forkSession	P6
실행	model, maxTurns, cwd, env, settingSources	P2, P6

SDK 권한 모드 5종

무인 라이브러리 관점의 재배열

acceptEdits

파일 편집, 일반 FS 자동 승인

신뢰 개발 워크플로

dontAsk

allowedTools 밖은 전부 거부

잠금형 헤드리스

auto (TS 한정)

분류기가 호출별 승인, 거부

가드레일 자율 에이전트

bypassPermissions

전 도구 무확인

샌드박스 CI 한정

default

canUseTool 콜백이 승인 처리

커스텀 승인 UI

에이전트 루프

SDK가 대신 도는 사이클

THE AGENT LOOP

생각, 도구 호출, 결과 관찰, 다음 결정의 사이클을 SDK가 관리합니다.

우리는 스트림을 소비하고, 루프는 완료나 오류에서 끝납니다.

THINK

사고

assistant text 블록

ACT

도구 호출

tool_use, SDK가 실행

OBSERVE

결과 관찰

tool_result 회신

DECIDE

종료 판단

완료 시 result 봉투

버전과 마이그레이션 노트

발밀을 확인하는 두 가지

RENAME

패키지 개명

구 claude-code SDK에서 이주
, migration-guide 참조.

VERSION

SDK 최신 유지

신모델 사고 방식 변경 등은
SDK 갱신으로 흡수.

PIN

버전 고정

프로덕션은 lockfile로 고정,
갱신은 스테이징 검증 후.

DEMOS

공식 예제

anthropics/claude-agent-
sdk-demos 저장소.

migration-guide

이주 문서

lockfile

재현성

demos repo

레퍼런스 구현

changelog

갱신 확인처

Part 1 Recap

SDK 기본 핵심 정리

KEY TAKEAWAY

query 하나가 에이전틱 루프 전체입니다.

options 여섯 서랍이 에이전트를 정의하고, 우리는 메시지 스트림의 소비자가 됩니다.

핵심 정리

1. 엔진 3 창구: CLI, -p, SDK
2. TS는 CC 바이너리 동봉
3. 인증: API 키 또는 USE_BEDROCK
4. 메시지 4류: system, assistant, user, result
5. 모드 5종, default는 콜백 필요

다음 파트

1. Part 2 쿼리와 멀티턴
2. 스트리밍 vs 단일 수집
3. continue, resume, fork
4. 시스템 프롬프트 옵션
5. 에러 처리 두 층

CHAPTER 06 / PART 02

쿼리와 멀티턴

스트림을 다루는 손

1. 스트리밍 vs 단일 수집
2. 멀티턴: continue, resume, fork
3. 시스템 프롬프트 옵션
4. 중단, 상한, 에러 두 층

스트리밍 vs 단일 수집

두 입력, 소비 모드

TWO MODES

진행을 실시간으로 보여줄지, 끝나고 결과만 쓸지가 기준입니다.

UI는 스트리밍, 배치와 CI는 단일 수집이 정석입니다.

스트리밍 소비

1. async for로 메시지 즉시 처리
2. 타자기 UI, 진행 표시에 필수
3. 도구 호출 관찰, 로깅 겸용
4. 멀티턴 스트리밍 입력도 지원

단일 수집

1. 루프를 돌리되 result만 취함
2. 백그라운드 잡, CI 파이프라인
3. 코드가 단순, 지연 무관 작업
4. Ch.5 -p 단발과 같은 감각

스트리밍 소비 패턴

블록 순회의 관용구

streaming (TS)

```
for await (const m of query({ prompt, options })) {
  if (m.type === "assistant" && m.message?.content) {
    for (const block of m.message.content) {
      if ("text" in block) ui.append(block.text);
      else if ("name" in block)
        ui.status(`Tool: ${block.name}`);
    }
  } else if (m.type === "result") {
    ui.done(m.subtype, m.total_cost_usd);
  }
}
```

블록 순회

text / tool_use

상태 표시

도구명 노출

result 마감

비용 동봉

Python 동형

hasattr 분기

단일 수집 패턴

결과만 필요한 배치의 형태

collect (Python)

```
async def run(prompt: str, options) -> ResultMessage:
    final = None
    async for m in query(prompt=prompt, options=options):
        if isinstance(m, ResultMessage):
            final = m
    if final is None or final.subtype != "success":
        raise RuntimeError(f"agent failed: {final and final.subtype}")
    return final

res = await run("의존성 취약점을 정리해", opts)
print(res.result, res.total_cost_usd)
```

result만

취득 대상

subtype 판정

성패 게이트

함수화

서비스 빌딩블록

usage 동봉

회계 재료

멀티턴 / 세션 이어가기

continue, resume, fork 옵션

multi-turn options

```
# 1턴: 세션 ID 확보
res = await run("이 저장소를 분석해 계획을 세워", opts)
sid = res.session_id

# 2턴: 같은 맥락에서 계속
opts2 = ClaudeAgentOptions(resume=sid, ...)
await run("계획의 1단계를 구현해", opts2)

# 변형
# continue_conversation=True : 최근 대화 이어받기
# fork_session=True          : 원본 보존 분기 (A/B)
# Ch.5의 -c, -r, --fork-session과 동일 의미론
```

session_id
result 봉투에서

resume 옵션
특정 세션 재개

fork
원본 보존 분기

Ch.5 동형
CLI 플래그 대응

대화형 클라이언트

불박이 세션이 필요한 제품

INTERACTIVE CLIENT

챗 UI처럼 사용자가 이어서 말을 거는 제품은 세션을 붙들고 스트리밍 입력을 주고받는 클라이언트 패턴을 씁니다.

Python 레퍼런스의 클라이언트 클래스가 이를 담당합니다.

WHEN

적용 대상

챗봇, IDE 보조 등 왕복 대화 제품.

HOW

구성

연결 유지 + 메시지 송신 +
응답 스트림 수신.

VS

query와 관계

단발, 배치는 query, 상주
왕복은 클라이언트.

REF

레퍼런스

python, typescript API
문서의 클라이언트 절.

상주 왕복

클라이언트의 영역

스트리밍 입력

지원 모드

query 우선

기본 선택지

API ref

시그니처 원전

시스템 프롬프트 옵션

정체성의 세 선택지

system prompt (TS)

// 1. 기본 유지 + 규칙 추가 (권장 출발점)

```
options: { systemPrompt: {  
  type: "preset", preset: "claude_code",  
  append: "모든 응답은 한국어. PEP 8 준수." } }
```

// 2. 완전 커스텀 (비코딩 에이전트)

```
options: { systemPrompt:  
  "너는 사내 위키 Q&A 도우미다. ..." }
```

// 3. 미지정: SDK 기본 프롬프트

// Ch.5의 append vs 대체 판단 기준 그대로

preset+append

정체성 유지

문자열 대체

책임 이전

CLAUDE.md 별개

로드는 P6

Ch.5 기준

판단 재사용

상한과 중단

루프의 고삐

TURNS

maxTurns

턴 상한, 초과 시 오류 결과. 무인 필수 (Ch.5 규율).

MODEL

model, effort

옵션으로 세션 모델과 노력 수준 지정.

CANCEL

취소

이터레이터 중단, AbortController류로 정리 종료.

CWD

cwd

에이전트 작업 루트 지정, 접근 범위의 기준.

maxTurns

폭주 방지

abort

정리 취소

cwd 기준

파일 접근 루트

예산은 상위

게이트웨이 (Ch.3)

에러 처리 두 층

루프의 죽음과 도구의 실패

TWO ERROR LAYERS

query 자체의 실패는 예외로 터지고, 도구의 실패는 `isError` 결과로 루프 안에서 소화됩니다.

층을 구분해야 처방이 맞습니다.

루프 층 (query 실패)

1. 인증, 네트워크, 미처리 예외
2. try, catch로 감싸 재시도, 폴백
3. result subtype 오류류도 이 층에서 판정
4. Ch.5 재시도 골격 이식 가능

도구 층 (isError)

1. 핸들러가 잡아 `isError`로 반환 (P3)
2. Claude가 보고 재시도, 우회
3. 루프는 계속: 자기 치유의 원천
4. 미포착 throw는 루프 층으로 승격

Part 2 Recap

쿼리와 멀티턴 핵심 정리

KEY TAKEAWAY

UI는 스트리밍, 배치는 단일 수집.

세션 좌표로 멀티턴을 만들고, 에러는 루프 층과 도구 층을 구분해 처방합니다.

핵심 정리

1. 블록 순회 관용구
2. `run()` 단일 수집 빌딩블록
3. `resume`, `continue`, `fork` 옵션
4. `preset+append`가 출발점
5. `maxTurns` 무인 필수

다음 파트

1. Part 3 커스텀 도구
2. tool 4요소와 인프로세스 서버
3. TS Zod, Python 스키마
4. 에러 규약과 어노테이션
5. 사내 위키 예시

CHAPTER 06 / PART 03

커스텀 도구

내 함수가 Claude의 손이 된다

1. 인프로세스 MCP 서버
2. tool 4요소: TS Zod, Python 스키마
3. 가용성 vs 권한의 층 분리
4. 에러 규약, 어노테이션, 미디어 반환

왜 커스텀 도구인가

인프로세스 vs 외부 서버

IN-PROCESS MCP

SDK의 인프로세스 서버는 별도 프로세스 없이 내 앱 안에서 도구를 노출합니다.

앱 전용 로직은 인프로세스, 팀 공유 자산은 외부 서버가 제자리입니다.

인프로세스 (오늘의 주인공)

1. 함수 정의가 곧 도구, 배포물 없음
2. 앱의 DB 커넥션, 상태 직접 접근
3. 지연 최소화, 타입 공유
4. 이 앱 전용 도메인 로직

외부 MCP 서버 (Ch.4)

1. stdio, http로 별도 운영
2. 여러 클라이언트가 공유
3. 언어, 팀 독립 배포
4. 조직 공용 자산화 대상

tool의 4요소

이름, 설명, 스키마, 핸들러

FOUR PARTS

도구는 네 부품입니다.

이름은 호출 식별자, 설명은 Claude의 선택 근거, 스키마는 인자 계약, 핸들러는 검증된 인자를 받는 비동기 함수입니다.

NAME

이름

고유 식별자, `mcp__서버__`
이름으로 노출.

DESC

설명

언제 쓸지 Claude가 읽는
문장, 품질이 곧 발동률.

SCHEMA

입력 스키마

TS는 Zod, Python은 타입
dict 또는 JSON Schema.

HANDLER

핸들러

async 함수, content 배열
(+isError) 반환.

4 부품

정의의 전부

설명 = UX

발동률 좌우

검증 후 전달

스키마 효과

content 계약

반환 규약

정의 / TypeScript

Zod가 스키마이자 타입

tools.ts

```
import { tool, createSdkMcpServer } from "@anthropic-ai/claude-agent-sdk";
import { z } from "zod";

const getTemperature = tool(
  "get_temperature",
  "지정 좌표의 현재 기온을 조회",
  { latitude: z.number().describe("위도"),
    longitude: z.number().describe("경도") },
  async (args) => { // args: { latitude, longitude } 자동 타입
    const t = await fetchTemp(args.latitude, args.longitude);
    return { content: [{ type: "text", text: `기온: ${t}C` }] };
  });
```

Zod 형태

스키마 = 타입

.describe()

필드 설명 노출

.default()

옵션 인자화

자동 추론

args 타입

정의 / Python

@tool 데코레이터와 타입 dict

tools.py

```
from claude_agent_sdk import tool, create_sdk_mcp_server

@tool(
    "get_temperature",
    "지정 좌표의 현재 기온을 조회",
    {"latitude": float, "longitude": float},
)
async def get_temperature(args):
    t = await fetch_temp(args["latitude"], args["longitude"])
    return {"content": [
        {"type": "text", "text": f"기온: {t}C"}]}
# enum, 범위, 중첩이 필요하면 JSON Schema dict 직접 전달
```

타입 dict

간편 스키마

JSON Schema

정밀 스키마 대안

옵션 인자

스키마 제외 + args.get

동일 반환

content 계약

서버 래핑과 연결

mcpServers에 꽂고 이름으로 허용

wire-up (TS)

```
const weatherServer = createSdkMcpServer({
  name: "weather", version: "1.0.0",
  tools: [getTemperature, getPrecipitation]
});

for await (const m of query({
  prompt: "서울 지금 기온과 강수 확률은?",
  options: {
    mcpServers: { weather: weatherServer },
    allowedTools: ["mcp__weather__get_temperature",
                  "mcp__weather__get_precipitation"]
  }
})) { /* ... */ }
```

키 = 서버명

mcpServers 규칙

mcp__w__t

완전 이름

allowed 등재

무프롬프트 실행

Python 동형

mcp_servers dict

다도구와 컨텍스트

서버 하나에 몇 개까지

MANY

배열 확장

tools 배열에 계속 추가,
재래핑으로 갱신.

WILDCARD

와일드카드

mcp__weather__* 로 서버
전체 허용.

COST

컨텍스트 비용

도구마다 매 턴 정의가 실림
, 수십 개면 부담.

SCALE

규모 해법

수백, 수천 도구는 tool
search 온디맨드 (P6).

배열 = 서버
구성 단위

__* 허용
일괄 등재

턴당 과금
정의의 비용

tool search
Ch.4 재회

가용성 vs 권한의 층

tools와 allowed, disallowed의 분업

tools: ["Read","Grep"]

나열한 내장만 컨텍스트에 존재

가용성 층

tools: []

내장 전부 제거, 내 MCP 도구만

순수 커스텀 에이전트

allowedTools

등재 도구는 무프롬프트 실행

권한 층

disallowedTools

호출 거부, 단 컨텍스트에는 남음

시도 낭비 가능

원칙

내장 제한은 tools로 제거가 우선

안 보이면 안 시도

에러 규약

던지면 죽고, isError면 산다

error contract (TS)

```
async (args) => {
  try {
    const r = await fetch(args.endpoint);
    if (!r.ok) return {
      content: [{ type: "text",
        text: `API error: ${r.status}` }],
      isError: true };      // 루프 계속, Claude가 대응
    return { content: [{ type: "text",
      text: await r.text() }] };
  } catch (e) { return { content: [{ type: "text",
    text: `failed: ${e}` }], isError: true }; }
} // 미포착 throw = query 전체 실패
```

isError: true

루프 생존 반환

미포착 throw

query 사망

Claude 반응

재시도, 우회

Python is_error

동일 규약

도구 어노테이션

행동 힌트 메타데이터

<code>readOnlyHint</code>	부수효과 없음 선언, 기본 <code>false</code>	읽기 도구 병렬 배치 허용
<code>destructiveHint</code>	파괴 가능성, 기본 <code>true</code>	정보성
<code>idempotentHint</code>	재호출 무해, 기본 <code>false</code>	정보성
<code>openWorldHint</code>	외부 시스템 접촉, 기본 <code>true</code>	정보성
성격	힌트일 뿐 강제 아님	핸들러와 일치시킬 것

이미지와 리소스 반환

text를 넘는 결과 블록

media blocks

```
// 이미지: raw base64 + mimeType (URL 필드 없음)
return { content: [{ type: "image",
  data: buf.toString("base64"), // 접두어 없이
  mimeType: "image/png" }] };
// -> Claude가 시각 입력으로 처리

// 리소스: URI 라벨 + 내용 동봉
return { content: [{ type: "resource", resource: {
  uri: "file:///tmp/report.md", // 참조용 라벨
  mimeType: "text/markdown",
  text: "# Report ..." } }] };
```

3 블록

text, image, resource

raw base64

data: 접두 금지

시각 입력화

이미지 처리

uri = 라벨

SDK가 읽지 않음

외부 MCP 서버 결합

인프로세스와 stdio, http의 동거

● ● ● mixed servers

```
options: {
  mcpServers: {
    wiki: wikiServer,           // 인프로세스
    github: { type: "http",     // 원격
      url: "https://api.githubcopilot.com/mcp/" },
    db: { type: "stdio",        // 로컬 스폰
      command: "npx", args: ["-y", "@corp/db-mcp"] }
  },
  allowedTools: ["mcp__wiki__*",
    "mcp__github__get_issue"]
} // Ch.4의 .mcp.json 문법이 옵션 객체로
```

동거 가능

3 형태 혼합

Ch.4 문법

type, url, command

이름 일원화

mcp__ 체계 공통

OAuth 서버

사전 인증 필요

실전 / 사내 위키 검색 도구

Ch.4 P6의 완성

wiki tool (TS)

```
const searchWiki = tool(  
  "search_wiki",  
  "사내 위키에서 문서를 검색해 상위 결과를 반환",  
  { query: z.string().describe("검색어"),  
    limit: z.number().int().max(10).default(5) },  
  async (args) => {  
    const rows = await wikiIndex.search(args.query, args.limit);  
    if (rows.length === 0) return { content: [{ type: "text",  
      text: "검색 결과 없음" }], isError: false };  
    return { content: [{ type: "text",  
      text: rows.map(r => `${r.title}: ${r.snippet}`).join("\n") }],  
      annotations: { readOnlyHint: true } } };
```

내부 인덱스

앱 상태 직접 접근

readOnly

병렬 배치 허용

무결과 처리

명시 텍스트

P8 재사용

실전 프로젝트 부품

Part 3 Recap

커스텀 도구 핵심 정리

KEY TAKEAWAY

4요소로 정의하고 서버로 묶어 옵션에 꽂습니다.

가용성은 tools, 권한은 allowed로 층을 가르고, 실패는 isError로 루프 안에 남기세요.

핵심 정리

1. 앱 전용은 인프로세스, 공유는 외부
2. 설명 품질 = 발동률
3. tools 제거 우선 원칙
4. throw는 사망, isError는 생존
5. readOnlyHint로 병렬

다음 파트

1. Part 4 구조화 출력
2. output_format 옵션
3. Zod, Pydantic 타입 세이프
4. 다단 도구 + 스키마
5. 오류 subtype 처리

구조화 출력

자유 텍스트에서 타입 계약으로

1. `output_format`과 `structured_output`
2. Zod, Pydantic 타입 세이프
3. 다단 도구 사용과 스키마의 결합
4. 오류 `subtype`과 설계 팁

왜 구조화 출력인가

파싱 코드의 소멸

FROM TEXT TO CONTRACT

에이전트가 어떤 도구를 얼마나 쓰든, 마지막에는 내 스키마에 맞는 검증된 JSON이 돌아옵니다.
불일치 시 SDK가 재프롬프트로 교정합니다.

PROBLEM

자유 텍스트

레시피, 계획을 정규식으로
긁던 취약한 파서.

SOLUTION

스키마 계약

형태를 선언하면 검증된
JSON만 수신.

RETRY

자동 교정

불일치 시 재프롬프트,
한도 초과면 오류 subtype.

CH.5

CLI 대응

--json-schema의 라이브러리 판

검증 보증

수신 데이터 성격

재프롬프트

SDK의 교정 루프

파서 제거

코드 다이어트

UI 직결

타입 데이터 활용

output_format 옵션

선언하고, structured_output으로 받는다

quick start (Python)

```
schema = {"type": "object", "properties": {
    "company_name": {"type": "string"},
    "founded_year": {"type": "number"}},
    "required": ["company_name"]}

async for m in query(
    prompt="Anthropic을 조사해 핵심 정보를 정리해",
    options=ClaudeAgentOptions(
        output_format={"type": "json_schema",
            "schema": schema})):
    if isinstance(m, ResultMessage) and m.structured_output:
        print(m.structured_output)  # 검증된 dict
```

json_schema

type 고정값

structured_output

결과 필드

TS outputFormat

카멜 표기

표준 기능 지원

enum, \$ref, 중첩

타입 세이프 / Zod

스키마 한 벌로 검증과 타입까지

zod (TS)

```
const FeaturePlan = z.object({
  feature_name: z.string(),
  steps: z.array(z.object({
    description: z.string(),
    complexity: z.enum(["low", "medium", "high"]) })),
  risks: z.array(z.string()) });

options: { outputFormat: { type: "json_schema",
  schema: z.toJSONSchema(FeaturePlan) } }
// 수신 후
const parsed = FeaturePlan.safeParse(m.structured_output);
if (parsed.success) usePlan(parsed.data); // 완전 타입
```

toJSONSchema

스키마 생성

safeParse

런타임 재검증

z.infer 타입

코드 전파

P3 재활용

도구 스키마와 한 벌

타입 세이프 / Pydantic

모델 클래스가 곧 계약

pydantic (Python)

```
from pydantic import BaseModel

class Step(BaseModel):
    description: str
    complexity: str # low, medium, high

class FeaturePlan(BaseModel):
    feature_name: str
    steps: list[Step]
    risks: list[str]

# output_format schema = FeaturePlan.model_json_schema()
# 수신: plan = FeaturePlan.model_validate(m.structured_output)
```

model_json_schema

스키마 생성

model_validate

타입 객체화

IDE 자동완성

필드 접근

재사용 모델

API 응답과 공유

다단 도구 + 스키마

탐색은 자유, 결말은 계약

todo agent

```
# 에이전트가 Grep으로 TODO를 찾고 Bash로 git blame까지
# 스스로 수행한 뒤, 마지막에 스키마로 응답
todo_schema = {"type": "object", "properties": {
  "todos": {"type": "array", "items": {"type": "object",
    "properties": {"text": {"type": "string"},
    "file": {"type": "string"}, "line": {"type": "number"},
    "author": {"type": "string"}},
    "required": ["text", "file", "line"]}},
  "total_count": {"type": "number"}},
  "required": ["todos", "total_count"]}
# author처럼 없을 수 있는 정보는 optional로
```

도구 자유

Grep, Bash 자율

결말 계약

스키마 응답

optional 설계

불확실 필드

P3 결합

커스텀 도구와도

오류 subtype과 설계 팁

계약이 깨질 때

ERROR HANDLING

재시도 한도 안에서 유효 출력을 못 만들면 subtype이 error_max_structured_output_retries로 옵니다.
스키마 단순화가 최고의 예방입니다.

실패 처리

1. subtype == success && structured_output 확인
2. retries 오류: 폴백 경로 준비
3. 폴백: 스키마 축소 재시도, 비정형 수신
4. 실패율을 관측 지표로 (P7)

예방 설계 팁

1. 스키마는 좁고 알게 시작
2. 없을 수 있는 필드는 optional
3. 프롬프트와 스키마의 의도 일치
4. enum으로 판정 흔들림 억제

Part 4 Recap

구조화 출력 핵심 정리

KEY TAKEAWAY

탐색은 자유롭게, 결말은 스키마로. Zod와 Pydantic 한 벌이 검증과 타입을 끝내고, 실패는 subtype으로 판정합니다.

핵심 정리

1. output_format + structured_output
2. 재프롬프트 자동 교정
3. toJSONSchema, model_json_schema
4. safeParse, model_validate
5. 좁은 스키마 + optional

다음 파트

1. Part 5 권한과 혹
2. 통제 4수단 지형
3. canUseTool 콜백
4. SDK 혹 콜백
5. 평가 순서

권한과 혹

코드로 주는 권한과 혹

1. 통제 4수단 지형
2. canUseTool 콜백
3. SDK 혹 콜백
4. 평가 순서와 승인 UI

통제 4수단 지형

무엇으로 무엇을 막는가

모드 + 규칙

permissionMode, allowed, disallowed

선언형 기본선

가용성

tools 옵션의 노출 제거

P3의 층 분리

canUseTool

호출별 승인을 내 함수가 결정

동적, 문맥 판단

혹

수명주기 지점의 콜백 개입

감사, 변조, 차단

조합 원칙

선언으로 8할, 콜백은 회색 지대

단순한 것이 강하다

canUseTool 콜백

승인 결정이 내 함수로 온다

canUseTool (TS)

```
options: {
  permissionMode: "default", // 콜백 필요 모드
  canUseTool: async (toolName, input) => {
    if (toolName === "Bash" &&
        /rm -rf|drop table/i.test(input.command ?? ""))
      return { behavior: "deny",
              message: "파괴적 명령 차단" };
    if (toolName.startsWith("mcp__wiki__"))
      return { behavior: "allow", updatedInput: input };
    return await askHuman(toolName, input); // UI 위임
  } }
```

default 모드

콜백 발동 조건

allow / deny

behavior 반환

updatedInput

입력 변조 가능

askHuman

승인 UI 접점

선언형 규칙 옵션

Ch.4 문법이 옵션으로

declarative rules

```
options: {
  permissionMode: "dontAsk",      // 무인 잠금
  tools: ["Read", "Grep", "Bash"], // 내장 노출 제한
  allowedTools: [
    "Read", "Grep",
    "Bash(npm run test *)",      // 규칙 문법 그대로
    "mcp__wiki__*"
  ],
  disallowedTools: ["Bash(curl *)"]
}
# 규칙 문법, deny 우선은 Ch.4 Part 2와 동일
```

규칙 문법

Tool(specifier)

deny 우선

평가 원칙 동일

dontAsk 짝

allow = 능력

settings 병합

P6 로드 시

SDK 혹은 콜백

Ch.4의 훅이 함수가 되다

hooks option (TS)

```
options: { hooks: {
  PreToolUse: [{ matcher: "Bash", hooks: [
    async (input) => {
      audit.log(input.tool_name, input.tool_input);
      if (isDangerous(input.tool_input.command))
        return { decision: "block",
                  reason: "guard: dangerous command" };
      return {}; // 무결정 통과
    } ] }],
  PostToolUse: [{ matcher: "Edit|Write",
    hooks: [ formatAfterEdit ] } ]
} }
```

함수 핸들러
스크립트 대신

이벤트, 매처
Ch.4 체계 동일

block 반환
차단 결정

인프로세스
앱 상태 접근

canUseTool vs 후

겹쳐 보이는 둘의 분업

SCOPE

범위

콜백은 승인 결정 전용, 후는
30 이벤트 전 수명주기.

TIMING

시점

콜백은 확인 필요 시점,
후는 매칭 이벤트마다.

USE 1

콜백 적합

사용자 승인 UI,
문맥 기반 동적 정책.

USE 2

후 적합

감사, 후처리, 알림, 환경
반응.

승인 전용
콜백의 정체

수명주기 전반
후의 영역

병행 가능
역할 분담

선언 우선
공통 전제

승인 UI 패턴

user-input: 사람을 루프에 넣기

STEP 1

확인 발생

default 모드, 미허용 호출

STEP 2

콜백 수신

canUseTool
(toolName, input)

STEP 3

UI 표시

웹, 슬랙 버튼으로 사람에게

STEP 4

결정 회신

allow, deny를 SDK로 반환

명확화 질문 등 사용자 입력 중계도 같은 결의 패턴 (user-input 문서)

human in loop

패턴의 이름

비동기 대기

콜백이 UI 대기

타임아웃 설계

무응답 기본값

P8 적용

실전 프로젝트

평가 순서 정리

한 호출이 지나는 관문들

1 가용성

tools에 없으면 시도 자체가 없음

2 혹 PreToolUse

block 결정 시 여기서 종료

3 규칙

disallowed 거부, allowed 통과

4 모드, 콜백

무일치 호출의 기본 거동, default는 canUseTool

5 실행과 Post 혹

통과 시 실행, 사후 혹 발화

Part 5 Recap

권한과 혹 핵심 정리

KEY TAKEAWAY

선언으로 8할을 막고, 승인은 canUseTool, 수명주기는 혹 콜백으로. 다섯 관문의 평가 순서가 진단의 지도입니다.

핵심 정리

1. 4수단: 규칙, 가용성, 콜백, 혹
2. default 모드 = 콜백 필수
3. 혹 핸들러가 함수로
4. 승인 UI 패턴 + 타임아웃
5. 5 관문 평가 순서

다음 파트

1. Part 6 세션, 상태, CC 기능
2. 세션 영속과 외부 스토리지
3. 비용 추적
4. settingSources와 스킬 로드
5. 서브에이전트 옵션

CHAPTER 06 / PART 06

세션, 상태, CC 기능

엔진의 기억과 자산을 코드에서

1. 세션 영속과 외부 스토리지
2. 체크포인트와 비용 추적
3. settingSources: Ch.1-4 자산 로드
4. 서브에이전트, 스킬, 도구 검색

세션 옵션 총정리

Part 2 좌표 체계의 사전판

<code>result.session_id</code>	매 실행이 남기는 좌표	저장해 후속예 사용
<code>resume=sid</code>	그 좌표에서 재개	다단 워크플로
<code>continue_conversation</code>	최근 대화 이어받기	cwd 기준
<code>fork_session=True</code>	원본 보존 분기	A/B 실험
트랜스크립트	~/.claude/projects JSONL	Ch.5 P3와 동일 집

세션 외부 스토리지

어느 호스트에서든 재개 가능하게

EXTERNAL SESSION STORAGE

기본 저장은 로컬 디스크입니다.

다중 호스트 서비스는 트랜스크립트를 S3나 Redis 같은 외부 저장소로 미러링해, 어떤 워커든 세션을 재개하게 만듭니다.

PROBLEM

로컬의 한계

워커 A의 세션을 워커 B가 모름.

MIRROR

미러링

트랜스크립트를 S3, Redis, 자체 백엔드로 동기.

RESUME

임의 재개

재개 전 로컬로 복원 후 resume.

DOC

원전

agent-sdk/session-storage 문서의 표준 패턴.

S3, Redis

대표 백엔드

any host

재개 자유

P7 전제

스케일링 기반

TTL 설계

보존 정책 병행

파일 체크포인트

SDK판 시간 여행

TRACK

변경 추적

에이전트 세션의 파일 편집
을 체크포인트로 기록.

RESTORE

지점 복원

이전 상태로 파일 되돌리기,
실험 실패의 안전망.

USE

서비스 활용

사용자에게 되돌리기 버튼
제공 가능.

LIMIT

한계

Bash 부수효과,
외부 시스템은 범위 밖
(Ch.5).

file-checkpointing

원전 문서

undo 버튼

제품화 포인트

git 병행

이력의 본진

Ch.5 동형

/rewind의 SDK판

비용 추적

봉투의 usage를 회계로

cost tracking

```
if isinstance(m, ResultMessage):  
    meter.record(  
        tenant=req.tenant_id,  
        cost=m.total_cost_usd,  
        turns=m.num_turns,  
        tokens=m.usage) # input, output, cache 필드
```

```
# 운영 포인트  
# 테넌트, 기능 태그로 절단해 대시보드화 (P7)  
# 프롬프트 캐싱 설정으로 입력 비용 절감  
# 상한: maxTurns + 상위 게이트웨이 한도 (Ch.3)
```

usage 필드

토큰 분해

테넌트 절단

과금 귀속

캐시 활용

입력 절감

cost-tracking

원전 문서

CC 자산 로드 / settingSources

Ch.1-4의 자산이 SDK로

load claude code features

```
options: {  
  settingSources: ["project"], // .claude 설정 로드  
  cwd: "/srv/checkout/payments"  
}  
// 로드되는 것: CLAUDE.md, settings의 permissions,  
// hooks, skills, commands, .mcp.json, agents/  
  
// 미지정 기본: 아무 설정도 읽지 않는 격리 실행  
// -> 서비스 재현성에 유리, 필요한 것만 명시 로드  
// Ch.5의 --setting-sources와 동일 의미론
```

명시 로드

기본은 격리

CLAUDE.md

프로젝트 규범 주입

skills, hooks

자산 재사용

재현성

서비스 관점 기본

서브에이전트 옵션

Ch.2의 위임을 코드로

agents option

```
options: {
  agents: {
    "code-reviewer": {
      description: "diff를 리뷰하는 전담 워커",
      prompt: "너는 시니어 리뷰어다. Critical과 ...",
      tools: ["Read", "Grep"],
      model: "sonnet"
    }
  }
}
// 파일 정의(agents/)는 settingSources로,
// 동적 정의는 이 옵션으로: Ch.5 --agents와 동형
// 병렬 위임, 컨텍스트 격리 효과는 Ch.2 그대로
```

인라인 정의

frontmatter 동형

파일 병행

settingSources 경로

격리 효과

Ch.2 의미론

prompt 필드

본문 대체

확장 자산 요약

스킬, 커맨드, 플러그인, 도구 검색

Skills	SKILL.md 자산을 세션에 로드	agent-sdk/skills
Slash commands	커맨드를 SDK 세션에서 실행	agent-sdk/slash-commands
Plugins	플러그인 묶음 로드	agent-sdk/plugins
Tool search	수천 도구의 온디맨드 로드	agent-sdk/tool-search
Todo tracking	작업 목록 이벤트로 진행 UI	agent-sdk/todo-tracking

Part 6 Recap

세션, 상태, CC 기능 핵심 정리

KEY TAKEAWAY

세션은 좌표와 외부 미러로 자유로워지고, 비용은 봉투로 회계가 되며, settingSources 한 줄이 Ch.1-4 자산 전부를 SDK로 데려옵니다.

핵심 정리

1. session_id 좌표 + resume, fork
2. S3, Redis 미러 = any host
3. usage 테넌트 절단
4. 기본 격리, 명시 로드
5. agents 인라인 = Ch.2 동형

다음 파트

1. Part 7 호스팅과 프로덕션
2. 서브프로세스 아키텍처
3. 컨테이너와 스케일링
4. 관측과 격리
5. 체크리스트

CHAPTER 06 / PART 07

호스팅과 프로덕션

에이전트를 인프라에 심기

1. 서브프로세스 아키텍처
2. 컨테이너와 스케일링
3. OTel 관측과 멀티테넌트 격리
4. 프로덕션 체크리스트

서브프로세스 아키텍처

SDK가 실제로 도는 방식

내 서비스

API 서버, 워커: query 호출과 스트림 소비

SDK

옵션 해석, 메시지 중계, 콜백 혹은 실행

CC 프로세스

에이전트 루프, 도구 실행 (서브프로세스)

모델 API

Anthropic 또는 Bedrock (Ch.3 경로)

컨테이너화

Dockerfile의 요점

Dockerfile

```
FROM node:22-slim
RUN apt-get update && apt-get install -y \
    git ripgrep ca-certificates && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
RUN useradd -m agent && chown -R agent /app
USER agent                # 비루트 실행
ENV CLAUDE_CODE_USE_BEDROCK=1 AWS_REGION=ap-northeast-2
CMD ["node", "server.js"]
```

node + git
런타임 필수품

비루트
격리 기본기

env 인증
키 미포함 이미지

작업 디렉토리
cwd 마운트 설계

Kubernetes 포인트

파드로 나갈 때 챙길 것

RES

자원 요청

서브프로세스 감당한 CPU,
메모리 여유와 limits.

IRSA

자격

IRSA, Pod Identity로
Bedrock 역할 (Ch.3).

STATE

무상태화

세션은 외부 스토리지 미러
(P6), 파드는 소모품.

PROBE

프로브

liveness는 프로세스,
readiness는 의존성 점검.

limits 필수

폭주 봉쇄

0 secrets

IRSA 인증

소모품 파드

재기동 자유

gateway 병행

Ch.3 조직 경로

스케일링

동시성의 세 다이얼

SCALING

쿼리 하나가 프로세스 하나라는 감각으로 동시성을 잡습니다. 워커 수, 큐, 세션 외부화의 세 다이얼이면 수평 확장이 성립합니다.

다이얼

1. 프로세스당 동시 쿼리 수 제한
2. 큐로 버스트 흡수, 워커 풀 소비
3. 세션 외부화로 워커 무상태화
4. 장시간 작업은 잡 큐 + 콜백 통지

수치 감각

1. 에이전트 1건 = 수십 초에서 수 분
2. 동기 API보다 잡 모델이 자연스러움
3. p95 실행 시간으로 워커 수 산정
4. 상한: maxTurns가 개별 폭주 봉쇄

관측 / OpenTelemetry

Ch.3 계측이 SDK에도

● ● ● observability

```
# 컨테이너 환경변수 (Ch.3 표준 세트)
CLAUDE_CODE_ENABLE_TELEMETRY=1
OTEL_METRICS_EXPORTER=otlp
OTEL_LOGS_EXPORTER=otlp
OTEL_EXPORTER_OTLP_ENDPOINT=http://otel-collector:4317
OTEL_RESOURCE_ATTRIBUTES=service=wiki-agent,env=prod

# 앱 층 보강
# 요청 ID를 로그, result 봉투 usage와 조인
# 구조화 실패율, 도구 에러율을 지표화 (P4, P3)
# agent-sdk/observability 문서가 원전
```

동일 세트

Ch.3 변수 재사용

서비스 속성

RESOURCE_ATTRIBUTES

2층 관측

SDK 지표 + 앱 지표

usage 조인

요청 단위 회계

격리와 보안 배포

secure-deployment의 3축

ISOLATION

에이전트는 코드를 실행하는 워크로드입니다.

실행 격리, 자격 최소화, 네트워크 통제의 3축을 테넌트 단위로 세웁니다.

격리 설계

1. 테넌트별 컨테이너, 작업 디렉토리 분리
2. 샌드박스, gVisor류 런타임 검토
3. tools 축소 + dontAsk로 능력 최소화
4. 쓰기 경로를 화이트리스트로

자격과 네트워크

1. IRSA 최소 권한, 테넌트 자격 미공유
2. 이그레스 도메인 통제 (Ch.3 P4)
3. 비밀은 핸들러 주입, 프롬프트 미노출
4. 감사: 혹 로그 + OTel 상관

Bedrock 프로덕션 연결

우리 조직의 표준 경로

bedrock wiring

```
# 컨테이너 env
CLAUDE_CODE_USE_BEDROCK=1
AWS_REGION=ap-northeast-2
ANTHROPIC_MODEL=apac.anthropic.claude-sonnet-5-v1:0

# 자격: IRSA 역할 (키 없음)
# bedrock:InvokeModel* 2종 + 리소스 한정 (Ch.3)

# 조직 게이트웨이 경유 시
# ANTHROPIC_BASE_URL=https://claude-gw.corp.example
# SSO, 한도, 감사가 게이트웨이에서 (Ch.3 P3)
```

env 3종
연결의 전부

IRSA
무키 자격

gateway 선택
조직 통제 경로

리전 프로파일
apac 추론 프로파일

프로덕션 체크리스트

출항 전 여덟 칸

GO-LIVE CHECK

상한, 격리, 세션, 관측의 네 축 여덟 칸입니다.

전부 예가 되면 출항입니다.

GUARD

상한과 통제

maxTurns, dontAsk
allowlist, 게이트웨이 한도.

ISOLATE

격리

비루트, 테넌트 분리, 이그레스
통제.

STATE

상태

세션 외부 미러, 재시도 먹등성,
TTL.

OBSERVE

관측

OTel 배선, 비용 절단,
실패율 알람.

4축 8칸

점검 구조

부하 검증

p95 산정 포함

롤백 경로

버전 고정 (P1)

런북

P8 운영 문서

Part 7 Recap

호스팅과 프로덕션 핵심 정리

KEY TAKEAWAY

쿼리는 서브프로세스입니다.

컨테이너에 여유를 주고, 세션을 외부화해 무상태로 만들고, Ch.3의 자격과 관측 체계를 그대로 얹으면 프로덕션입니다.

핵심 정리

1. 서브프로세스 감각의 자원 설계
2. 비루트 + 무키 IRSA
3. 큐 + 워커 + 세션 외부화
4. OTel 재사용 + 앱 지표 2층
5. 격리 3축과 8칸 체크

다음 파트

1. Part 8 실전 프로젝트
2. 사내 위키 Q&A 서비스
3. 부품 총조립
4. API 노출과 승인 루프
5. 테스트와 운영

CHAPTER 06 / PART 08

실전 프로젝트

사내 위키 Q&A 에이전트 서비스

1. 설계: 부품의 총조립
2. 에이전트 코어와 API 노출
3. Ch.5 파이프의 서비스화
4. 테스트와 운영

설계 / 부품의 총조립

어느 파트가 어디에 들어가나

API (Fastify)

POST /ask, 잡 큐, 승인 웹훅 (P5 UI 패턴)

에이전트 코어

query + wiki 도구(P3) + 스키마(P4)

통제

tools 축소, dontAsk, 혹 감사 (P5)

상태

session_id 저장, S3 미러, usage 회계 (P6)

인프라

컨테이너, IRSA, OTel (P7 + Ch.3)

에이전트 코어

P2-P4 부품이 한 함수로

core.ts

```
const Answer = z.object({
  answer: z.string(),
  sources: z.array(z.string()),
  confidence: z.enum(["low", "medium", "high"]) });
export async function ask(q: string, sid?: string) {
  for await (const m of query({ prompt: q, options: {
    systemPrompt: "너는 사내 위키 Q&A 도우미다. ...",
    tools: [], mcpServers: { wiki: wikiServer },
    allowedTools: ["mcp__wiki__*"],
    permissionMode: "dontAsk", maxTurns: 8, resume: sid,
    outputFormat: { type: "json_schema", schema: z.toJSONSchema(Answer) } } }))) {
    if (m.type === "result") return m; } }
```

tools: []
순수 커스텀

dontAsk + 8턴
무인 규율

resume 좌표
후속 질문

Answer 계약
confidence 동봉

API 노출

잡 모델과 승인 루프

server.ts (요점)

```
app.post("/ask", async (req, rep) => {
  const job = jobs.enqueue(req.body.q, req.body.session_id);
  return rep.code(202).send({ job_id: job.id });
}); // 실행은 워커 풀이 소비 (P7 잡 모델)

app.get("/jobs/:id", async (req) => jobs.status(req.params.id));
// done이면 { answer, sources, confidence, session_id }

// 승인 루프가 필요한 변형 (쓰기 도구 추가 시)
// canUseTool -> 슬랙 버튼 webhook -> resolve (P5)
// 회계: result.usage를 테넌트로 기록 (P6)
```

202 + 잡
비동기 계약

폴링, 웹훅
결과 회수

승인 변형
쓰기 확장 대비

usage 기록
요청 회계

Ch.5 파이프의 서비스화

pr-review.sh가 상주 서비스로

review-service (요점)

```
// Ch.5의 스키마와 게이트 로직을 SDK로 이식
export async function reviewPr(diff: string) {
  for await (const m of query({
    prompt: `다음 diff를 리뷰: 심각도, 사유, 차단\n${diff}`,
    options: {
      allowedTools: ["Read", "Grep"],
      permissionMode: "dontAsk", maxTurns: 12,
      outputFormat: { type: "json_schema",
        schema: reviewSchema } } }))) {
    if (m.type === "result") return m.structured_output;
  } } // 웹훅 수신 -> reviewPr -> 코멘트 게시
```

셀 -> SDK
이식의 방향

동일 스키마
Ch.5 자산 재사용

웹훅 상주
CI 밖의 선택지

다레포 서비스
조직 공용화

테스트 전략

에이전트를 어떻게 검증하나

TESTING

핸들러는 단위로, 에이전트는 계약으로 검증합니다.

자유 텍스트가 아닌 스키마 필드를 단언하는 것이 흔들리지 않는 테스트를 만듭니다.

단위와 통합

1. 도구 핸들러: 순수 함수로 단위 테스트
2. isError 경로 필수 커버
3. 에이전트: 고정 프롬프트 + 스키마 필드 단언
4. confidence, block 같은 enum이 단언점

회귀와 평가

1. 대표 질문 세트로 골든 평가
2. 실패율, 비용, 토큰 수를 추세 관리
3. 모델, SDK 갱신 시 평가 재실행
4. 스테이징에서 p95 실측 (P7)

운영 런북

출항 후의 네 문서

DASH

대시보드

성공률, p95, 비용, 구조화
실패율 4지표.

ALARM

알람 기준

실패율, 예산 소진, 큐 적체의
임계.

PLAY

플레이북

모델 폴백, 워커 증설,
세션 복구 절차.

REVIEW

주간 회고

실패 큐 소화, 비용 절단,
프롬프트 개선.

4 지표

대시보드 최소선

임계 명문화

알람 계약

절차서

새벽의 나를 위해

개선 루프

주간 리듬

Part 8 Recap

실전 프로젝트 핵심 정리

KEY TAKEAWAY

발명이 아니라 조립이었습니다.

코어 함수 하나, 잡 API, 계약 테스트, 런북 네 문서면 사내 에이전트 서비스가 쉽니다.

핵심 정리

1. 코어 = query + 도구 + 스키마
2. API는 잡 모델 + 승인 변형
3. Ch.5 자산의 SDK 이식
4. 테스트는 스키마 필드 단언
5. 런북 4문서

다음 파트

1. Part 9 Recap & Labs
2. 챕터 요약과 FAQ
3. 실습 4종
4. 워크샵 6챕터 총정리
5. 다음 여정

CHAPTER 06 / PART 09

Recap & Labs

챕터 정리와 워크샵 완결

1. Chapter 6 핵심 요약과 FAQ
2. 실습 랩 4종
3. 워크샵 6챕터 총정리
4. After Workshop 로드맵

Chapter 6 핵심 요약

여덟 파트를 여섯 문장으로

QUERY

루프

query 하나가 에이전틱 루프,
우리는 스트림 소비자.

TOOLS

도구

4요소 + 인프로세스 서버,
throw 대신 isError.

SCHEMA

계약

output_format과 Zod,
Pydantic으로 타입 세이프.

CONTROL

통제

선언 8할 + canUseTool, 혹 콜백,
5관문 순서.

STATE

상태

세션 좌표, 외부 미러, usage 회계,
자산 로드.

SHIP

출항

서브프로세스 감각, 컨테이너,
격리 3축, 런북.

학습 목표 체크리스트

네 가지 목표 자가 점검

SELF CHECK

네 문장에 답할 수 있으면 목표 달성입니다.

막히면 표기된 파트로 복습하세요.

CHECK 1

루프 장악

메시지 4류와 options 6 설명
. (P1, P2)

CHECK 2

도구 제작

인프로세스 도구를 정의,
연결. (P3)

CHECK 3

계약 출력

Pydantic 스키마로 결과 수신
. (P4)

CHECK 4

프로덕션

통제, 세션, 컨테이너 조립.
(P5-P8)

4 Checks

목표 점검

Part 참조

복습 경로

Labs 4종

미완은 실습으로

조립 감각

판정 기준

FAQ / 자주 나오는 질문

현장 질문 여섯 가지

저수준 API와 뭐가 다른가요

도구, 루프, 컨텍스트가 내장

왜 SDK 슬라이드

-p와 SDK 중 무엇을 쓰나요

스크립트는 -p, 제품은 SDK

Ch.5와 분업

도구에서 예외 던져도 되나요

루프가 죽음, isError로 반환

P3 규약

CLAUDE.md가 왜 안 읽히죠

기본 격리, settingSources 명시

P6

auto 모드가 안 돼요

현재 TS 한정

P1 모드 표

구독 로그인 붙여도 되나요

서드파티 제공 불가 정책

P1 인증

Lab 1 / 첫 query와 계약

소요 20분

LAB 1 STEPS

```
# 본 랩은 워크샵 웹사이트에서 진행합니다
$ open https://github.com/whchoi98/claude-code-workshop
  -> ccw-hands-on-lab / 해당 챕터의 랩 문서 선택
# 사전 준비물, 단계별 절차, 검증 체크리스트가 웹 문서로 제공됩니다
# 로컬에는 이 덱과 인증 환경만 준비하시면 됩니다
```

20분

예상 소요

버그 수정

공식 예제 흐름

블록 관찰

스트림 감각

Pydantic

계약 전환

Lab 2 / 커스텀 도구

소요 25분

● ● ● LAB 2 STEPS

```
# 본 랩은 워크샵 웹사이트에서 진행합니다
$ open https://github.com/whchoi98/claude-code-workshop
  -> ccw-hands-on-lab / 해당 챕터의 랩 문서 선택
# 사전 준비물, 단계별 절차, 검증 체크리스트가 웹 문서로 제공됩니다
# 로컬에는 이 데크와 인증 환경만 준비하시면 됩니다
```

25분

예상 소요

4요소

정의 체험

생사 비교

throw vs is_error

tools: []

층 분리 체험

Lab 3 / 통제 콜백

소요 20분

● ● ● LAB 3 STEPS

```
# 본 랩은 워크샵 웹사이트에서 진행합니다
$ open https://github.com/whchoi98/claude-code-workshop
  -> ccw-hands-on-lab / 해당 챕터의 랩 문서 선택
# 사전 준비물, 단계별 절차, 검증 체크리스트가 웹 문서로 제공됩니다
# 로컬에는 이 데크와 인증 환경만 준비하시면 됩니다
```

20분

예상 소요

y/n 승인

human in loop

혹 2종

감사 + 후처리

관문 추적

P5 순서 체득

Lab 4 / 컨테이너 배포

소요 25분

● ● ● LAB 4 STEPS

```
# 본 랩은 워크샵 웹사이트에서 진행합니다
$ open https://github.com/whchoi98/claude-code-workshop
  -> ccw-hands-on-lab / 해당 챕터의 랩 문서 선택
# 사전 준비물, 단계별 절차, 검증 체크리스트가 웹 문서로 제공됩니다
# 로컬에는 이 데크와 인증 환경만 준비하시면 됩니다
```

25분

예상 소요

이미지 1개

P7 요점 적용

잡 계약

202 + 플링

usage 로그

회계 확인

워크샵 총정리 / 6챕터 지도

3일의 여정을 한 장으로

CH.1

기초와 컨텍스트

설치, CLAUDE.md, 메모리, 명령의 기본기.

CH.2

Subagents

격리 위임, 병렬 워커, 에이전트 팀.

CH.3

Admin Setup

조직 배포, 자격, gateway, 거버넌스, 관측.

CH.4

Settings

권한 문법, 혹 30이벤트, MCP, 커맨드.

CH.5

CLI Reference

플래그 전집, 헤드리스, 세션, CI 자동화.

CH.6

Agent SDK

커스텀 도구, 계약 출력, 호스팅, 서비스화.

After Workshop 로드맵

다음 분기의 실행 계획

WEEK 1-2

개인 정착

Ch.1, 4 자산을 내 저장소에

WEEK 3-6

팀 표준

Ch.2, 4 커밋 + Ch.5 CI 1종

WEEK 7-10

조직 배포

Ch.3 managed, gateway
협의

WEEK 11+

서비스화

Ch.6 에이전트 1종 출항

각 단계의 관문: 비용 대장, 실패 큐, 분기 회고 (Ch.5 운영 장부)

4 단계

분기 로드맵

1종부터

작게 출항

장부 2권

비용, 실패 큐

분기 회고

개선 루프

References

본 챕터 공식 문서 출처 (code.claude.com/docs/en/agent-sdk)

기본	overview, quickstart, agent-loop	Part 1, 2
도구	custom-tools, mcp, tool-search	Part 3, 6
계약	structured-outputs	Part 4
통제	permissions, hooks, user-input	Part 5
상태	sessions, session-storage, cost-tracking	Part 6
운영	hosting, observability, secure-deployment	Part 7, 8

Thank you!

Choi, WooHyung / Prin. Solutions Architect, AWS Korea

whchoi@amazon.com | linkedin.com/in/woohyungchoi

Workshop code: github.com/whchoi98/claude-code-workshop